МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №4

по дисциплине: «Безопасность систем баз данных»

по теме: «Защита БД студий дубляжа»

Выполнил:

Студент гр. «АБс-322», «АВТФ»

Бутаков Никита Максимович

«___» _____ 2026г

_____

(подпись)

Проверил:

Ассистент кафедры ЗИ

Питько Яна Андреевна

«___» _____ 2026г

_____

(подпись)

Новосибирск 2026

Содержание

Введение 3
1. Подготовка инфраструктуры и шифрование WAL 4
2. Создание базовой резервной копии и моделирование аварии 6
3. Восстановление PITR из зашифрованных архивов 8
Вывод 10
Приложение А 12

Введение

Цели работы:

1. Освоить механизмы Point-in-Time Recovery (PITR) – восстановления базы данных на произвольный момент времени, что позволяет минимизировать потерю данных (RPO – Recovery Point Objective) при логических сбоях или действиях злоумышленника.

2. Обеспечить конфиденциальность резервных копий путем внедрения прозрачного шифрования архивов журнала предзаписи WAL (Write-Ahead Logging).

Тема: «Защита БД студий дубляжа – управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов».

Задачи:

1. Подготовка инфраструктуры и шифрование WAL;

2. Создание базовой резервной копии и моделирование аварии;

3. Восстановление PITR из зашифрованных архивов;

1. Подготовка инфраструктуры и шифрование WAL

Для того, чтобы наш кластер инициализировался с включенными контрольными суммами страниц данных мы в файл docker-compose.yml добавим параметр «POSTGRES_INITDB_ARGS: "--auth-host=scram-sha-256 --data-checksums"». Он задаёт безопасную аутентификацию для сетевых подключений, а --data-checksums включает контрольные суммы страниц данных для выявления физических повреждений на уровне хранения (Рисунок 1).

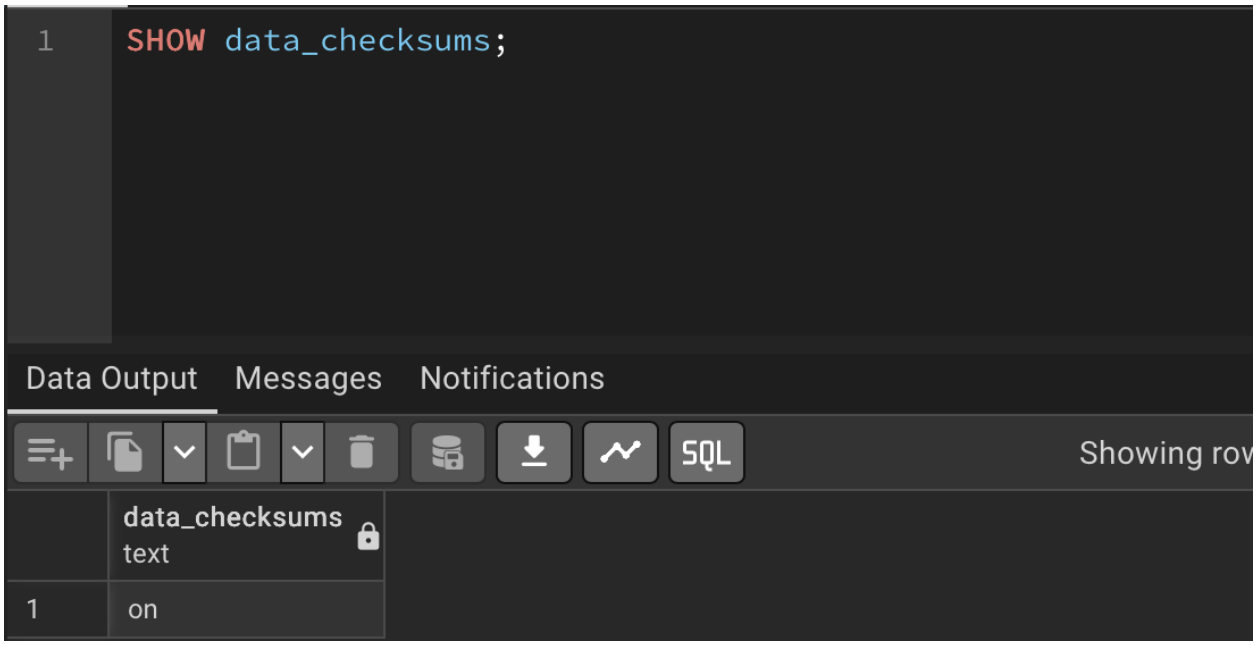


Рисунок 1 – Проверка контрольной суммы

В том же файле docker-compose.yml настроим postgresql.conf (Primary сервер): установим wal_level = replica и включим режим архивирования: archive_mode = on. Далее создадим локальную директорию /var/lib/postgresql/archive для хранения архивов и напишем скрипт для параметра archive_command (Рисунок 2).

```
-c wal_level=replica
-c archive_mode=on
-c archive_command='/usr/local/bin/s2_lr4_archive_wal.sh %f %p'
-c archive_timeout=60
```

Рисунок 2 – Параметры запуска в docker.compose.yml

Скрипт реализует безопасную команду архивирования WAL: принимает имя файла и путь к сегменту, сжимает его утилитой gzip и шифрует поток алгоритмом AES-256-CBC через openssl enc. Пароль для шифрования извлекается в приоритетном порядке: из переменной окружения WAL_ARCHIVE_PASS, из защищённого файла /var/lib/postgresql/secrets/wal_archive_pass.txt, либо используется fallback-значение. Результат сохраняется в директорию архива с двойным расширением .gz.enc, обеспечивая одновременно компрессию и конфиденциальность журналов. При любых проблемах доступа к исходному WAL-файлу скрипт немедленно завершается. Для применения настроек перезапустим сервер PostgreSQL.

Код скрипта представлен в Приложении А.

2. Создание базовой резервной копии и моделирование аварии

Создадим базовый бэкап, используя утилиту pg_basebackup для создания полной копии кластера. Также используем флаг, включающий проверку контрольных сумм, и сохраним бэкап в отдельную директорию /var/lib/postgresql/backup/base (Рисунок 3).

```
docker exec -u postgres "$CONTAINER" pg_basebackup \
    -D "$BACKUP_DIR" \
    -U postgres \
    -Fp -Xs -P \
    --checkpoint=fast
```

Рисунок 3 – Команда для базового бэкапа

Создадим схему pitr_lab и таблицу important_data, вставим в нее строку «Данные до аварии» (Рисунок 4).

```
CREATE TABLE pitr_lab.important_data (
    id SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO pitr_lab.important_data (message)
VALUES ('Данные до аварии');
```

	id [PK] integer	message text	created_at timestamp with time zone
1	1	Данные до аварии	2026-05-19 13:53:35.626757+...

Рисунок 4 – Таблица pitr_lab.important_data

Выполним команду, чтобы узнать актуальное время и запомним его (Рисунок 5).

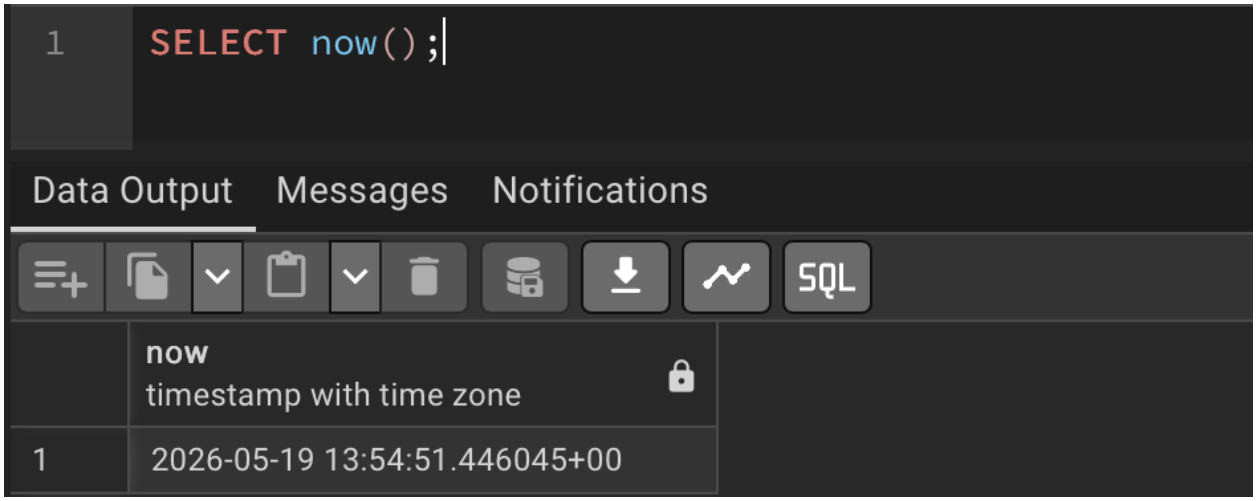


Рисунок 5 – Актуальное время

Далее подождем около 2-ух минут и внесем "разрушительные" изменения: удалим таблицу pitr_lab.important_data. После чего создадим эту же таблицу и сделаем запись: «Данные после аварии» и принудительно переключим сегмент WAL с помощью «SELECT pg_switch_wal();» (Рисунок 6).

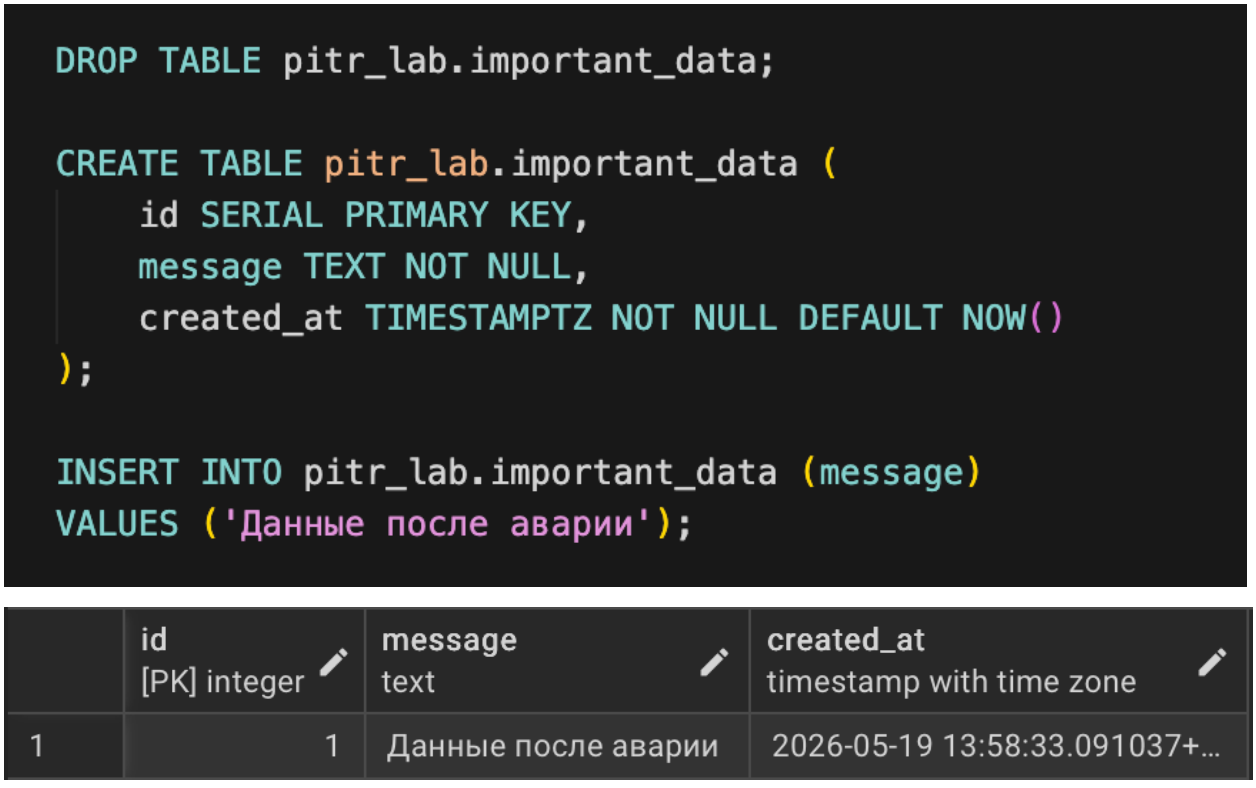


Рисунок 6 – Таблица pitr_lab.important_data после разрушения

3. Восстановление PITR из зашифрованных архивов

Остановим нашу службу PostgreSQL с помощью docker-compose stop и подготовим данные. Копируем PGDATA в /var/lib/postgresql/data_broken (Рисунок 7).

```
if [ -d "$PGDATA" ] && [ -n "$(ls -A "$PGDATA" 2>/dev/null || true)" ]; then
    BROKEN="${PGDATA}_broken_$(date +%Y%m%d%H%M%S)"
    mkdir -p "$BROKEN"
    cp -a "${PGDATA}/." "${BROKEN}/"
    find "$PGDATA" -mindepth 1 -delete 2>/dev/null || rm -rf "${PGDATA:?}"/*
```

Рисунок 7 – Копирование PGDATA

Развернем файлы из базового бэкапа в PGDATA (Рисунок 8).

```
mkdir -p "$PGDATA"
cp -a "${BACKUP_DIR}/." "${PGDATA}/"
```

Рисунок 8 – Развертывание базового бэкапа

Создадим файл recovery.signal и настроим параметры в postgresql.conf. Тут же мы установим параметр restore_command = '/usr/local/bin/s2_lr4_restore_wal.sh %f %p'. Данный скрипт расшифровывает файлы из архива и кладет их в папку pg_wal. В recovery_target_time мы записываем время, которое сохраняли до аварии. В recovery_target_action установим значение «promote» (Рисунок 9).

```
touch "${PGDATA}/recovery.signal"

mkdir -p "${PGDATA}/conf.d"
cat > "${PGDATA}/conf.d/99-pitr-recovery.conf" <<EOF
restore_command = '/usr/local/bin/s2_lr4_restore_wal.sh %f %p'
recovery_target_time = '${TARGET_TIME}'
recovery_target_action = 'promote'
EOF
```

Рисунок 9 – Параметры в postgresql.conf

Запустим сервер и проверим логи на наличие ошибок расшифровки (Рисунок 12).

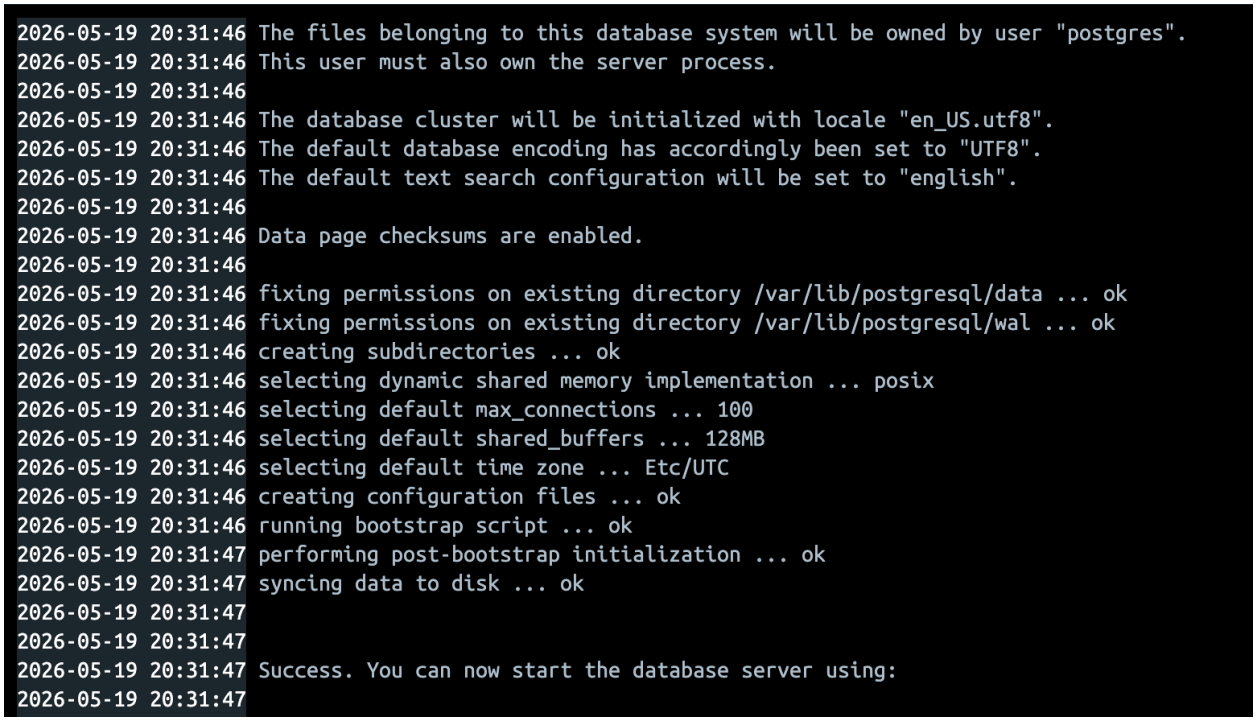


Рисунок 12 – Логи сервера

Подключимся к БД и проверим наличие таблицы important_data (Рисунок 13).

	id [PK] integer	message text	created_at timestamp with time zone
1	1	Данные до аварии	2026-05-19 13:53:35.626757+...

Рисунок 13 – Таблица pitr_lab.important_data после восстановления

Из рисунка 13 видим, что база данных восстановилась правильно и присутствует запись «До аварии».

Вывод

В ходе выполнения лабораторной работы для базы данных «студии дубляжа» (управление доступом к авторским материалам, контрактам с актёрами, расписаниям записей и защищённым архивам аудиофайлов) были реализованы три задачи по обеспечению безопасности и восстановлению данных.

В первом задании добавили параметр POSTGRES_INITDB_ARGS: "--auth-host=scram-sha-256 --data-checksums", что обеспечило контрольные суммы страниц данных для выявления физических повреждений. На основном сервере в postgresql.conf установлены wal_level = replica и archive_mode = on, создана директория для архивов. Написан скрипт архивации, который сжимает WAL-файлы через gzip, шифрует их алгоритмом AES-256-CBC через openssl enc, а ключ извлекает из переменной окружения или защищённого файла. В результате каждый архивный WAL-сегмент получает расширение .gz.enc, что обеспечивает конфиденциальность резервных копий при хранении.

Во втором задании с помощью утилиты pg_basebackup создана полная копия кластера с проверкой контрольных сумм в директорию /var/lib/postgresql/backup/base. Смоделировали «разрушительное» действие и выполнили принудительное переключение WAL-сегмента (pg_switch_wal()), чтобы все изменения попали в архив.

В третьем задании повреждённые данные PGDATA были скопированы в резерв. Базовый бэкап развёрнут обратно в PGDATA. Создан сигнальный файл recovery.signal и в postgresql.conf прописан restore_command – скрипт, расшифровывающий файлы из архива (обратный процесс: openssl enc -d и gunzip) и помещающий их в pg_wal. Указано целевое время восстановления (recovery_target_time), равное моменту до удаления таблицы, и действие recovery_target_action = promote. После запуска сервера логи подтвердили успешную расшифровку и применение WAL. Подключение к БД показало, что

таблица important_data восстановлена в состоянии «До аварии», а запись «После аварии» отсутствует.

Приложение А

```
set -euo pipefail

FNAME="${1:?}"
WALPATH="${2:?}"
ARCHIVE_DIR="${WAL_ARCHIVE_DIR:-/var/lib/postgresql/archive}"
OUT="${ARCHIVE_DIR}/${FNAME}.gz.enc"

if [ ! -r "$WALPATH" ]; then
   echo "s2_lr4_archive_wal: WAL not readable: $WALPATH" >&2
   exit 1
fi

PASS_FILE="${WAL_ARCHIVE_PASS_FILE:-
/var/lib/postgresql/secrets/wal_archive_pass.txt}"
if [ -n "${WAL_ARCHIVE_PASS:-}" ]; then
   PASS="$WAL_ARCHIVE_PASS"
elif [ -r "$PASS_FILE" ]; then
   PASS="$(tr -d '\r\n' < "$PASS_FILE")"
else
   PASS="MySecretPassword"
fi

mkdir -p "$ARCHIVE_DIR"
gzip -c "$WALPATH" | openssl enc -aes-256-cbc -salt -pass "pass:${PASS}" -out
"$OUT"
exit 0
```